

Homework 4 - Functions

Course: CIS 284 - Computer Programming 2 - Fall 2026

Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: Oct 29 at 11:59pm

Zip up both java files and upload to Canvas. **Make sure each file has your name at the top**

For 2 bonus points, answer the following questions (as a comment on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?
3. Was anything unclear in the directions? Do you have any other feedback?

Your responses will be helpful feedback for refining these assignments in future courses.

Assignment Overview

This assignment will help you practice organizing a larger program into functions. You will do this using the two programs from homework 3.

Grading Information

This homework emphasizes good program design, and so will be graded on more than just correctness. Pay close attention to the specific guidelines given in each part. You will not specifically be graded on requirements from earlier homework assignments, but your programs should still be functional. *If you can't complete everything, it is better to submit partial work than miss the deadline.*

Points	Name	Description
30	Effort	Do the programs compile and run? Was a reasonable attempt made?
5	Style	Is the code formatted well? Is it readable? Are variables named well?
10	Part 1	Does your program follow the design guidelines in Part 1?
5	Part 1	Are functions simulateShuffles and drawResults implemented correctly?
30	Part 2	Do your functions follow the design guidelines in Part 2?
5	Part 2	Does the program handle exceptions without crashing?
15	Part 2	Does the program implement remove , sum , and var correctly?

Note: Your output does not need to match my examples word for word. Feel free to add additional messages or personalize it, as long as it satisfies the requirements.

1. Shuffle Test

In this part, you will edit the **ShuffleTest** program from homework 3. Currently, your **ShuffleTest.java** program is likely one large main function. Note that there are three primary capabilities that are all intermixed:

1. Initializing variables and setting up the canvas
2. Running the simulation
3. Visualizing the simulation results

A general principle in good program design is that each function should operate at a particular level of abstraction, and functions should accomplish one clear task. (If you can't describe what a function does in a single sentence, it might be too complex).

Instructions

Rewrite your **ShuffleTest.java** program so that it follows the following guidelines:

1. Functions should not rely on any global variables. All needed inputs and outputs should be handled using parameters and return statements.
2. Give good descriptive names to your functions. Generally they should be an action or verb phrase.
3. Put a comment above each function summarizing what it does.
4. Your main function should only contain code to initialize things and orchestrate the simulation. It should not do any computations.

Your submission will be graded on how well it adheres to there guidelines.

To help guide you, I have provided you with these function signatures you must use:

```
// Performs 'n' shuffles on int arrays of length 'm' (values 0 to m-1),
//   and returns m*m array recording the frequency that item i ended in pos j
// If useGoodShuffle is true, runs shuffle1, otherwise, runs shuffle2
public static int[][] simulateShuffles(int m, int n, boolean useGoodShuffle)

// Given the simulation results as an m*m array,
// Visualizes them by drawing a 2D grid of squares,
//   where shade indicates frequency
public static void drawResults(int m, int n, int[][] results)
```

Create and implement these functions to reorganize **ShuffleTest.java**.

2. Stats (Again!)

In this part, you will edit the **Stats** program from homework 3. Currently, your **Stats.java** program is likely one large main function. Note that there are three primary capabilities that are intermixed:

1. Getting a command from the user and *deciding* what to do in response
2. Actually doing the specific action or task indicated
3. Doing specific mathematical calculations (max, mean, etc).

A general principle in good program design is that each function should operate at a particular level of abstraction, and functions should accomplish one clear task. (If you can't describe what a function does in a single sentence, it might be too complex).

Instructions:

Spend some time thinking about how to reorganize your **Stats.java** program and divide up the work into different functions. I recommend you plan things out on paper before starting to program.

Your program will be graded on the following guidelines:

1. Functions should not rely on any global variables. All needed inputs and outputs should be handled using parameters and return statements
 - The one exception: you may use a global scanner variable to read from standard in
2. Separate printing and calculations. For example, if you have a function that computes the max of an array, do not put print statements inside.
3. Give good descriptive names to your functions. Generally they should be an action or verb phrase.
4. Put a comment above each function summarizing what it does.
5. Avoid duplicate work.
 - If you are doing the same computation in different places, make it a function.
6. Your main function should only contain code directly related to managing the command loop.
 - **Tip:** for most commands, it should call a function to handle that command.

Note: If an exception is thrown when opening a file, your program *should not exit*. You should catch the exception and print an error message.

Additional Requirements:

The Stats program must implement the commands from the previous homeworks: help, load, save, print, min, max, mean, and add.

In addition, you should implement these commands:

1. **remove** - See Homework 2 for details
2. **sum** - Prints the sum of values in the array
3. **var** - Prints the variance of the values. (see next page)

Variance is given by the following function:

$$\text{Var}(X) = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

Where μ is the mean, x_i is each value in the array, and n is the length